

SOFTWARE VERIFICATION & VALIDATION LAB

Semester Project Report

Online Examination System

Submitted By (Group Members)

Muhammad Furqan

Roll Number: 032

Muhammad Monam Majeed

Roll Number:010

Abdul Rehman

Roll Number: 035

Section: A

Semester: 6th Semester

Program: BS Software Engineering

Course: Software Verification & Validation Lab

University: Lahore Garrison University

Submitted To:

Course Instructor (SVV Lab)

Submission Type:

Semester Project Report

1. Introduction

Software Verification and Validation (SVV) is an important field of software engineering that ensures software systems behave correctly according to specified requirements. Formal methods are mathematical techniques used to verify system behavior before implementation.

This project focuses on the formal verification of an Online Examination System using different verification methods and tools. The selected system contains clear states, operations, and constraints, making it suitable for formal modeling and validation.

The project demonstrates requirement engineering, formal specification, structural verification, and automated validation using Z Notation, VDM-SL, Alloy Analyzer, and GitHub Actions.

The main purpose of this project is not software development but verification of correctness through formal techniques.

2. Objectives

The objectives of this project are:

- To perform requirement engineering for an Online Examination System.
 - To identify ambiguities and inconsistencies in requirements.
 - To formally model the system using Z Notation.
 - To define functional specifications using VDM-SL.
 - To perform structural verification using Alloy Analyzer.
 - To validate system consistency using assertions and constraints.
 - To implement automated validation using GitHub Actions.
-

3. Methodology

The project follows a step-by-step Software Verification and Validation process.

Phase 1 — Requirement Engineering

System requirements were collected and documented in the Software Requirements Specification (SRS). Requirement defects such as ambiguity and inconsistency were identified and corrected.

Phase 2 — Z Notation Modeling

Formal state modeling was performed using Z Notation. System states, operations, and invariants were mathematically specified.

Phase 3 — VDM-SL Specification

Functional behavior was specified using VDM-SL with preconditions and postconditions.

Phase 4 — Alloy Verification

The structural relations of the system were verified using Alloy Analyzer.

Phase 5 — Validation

Validation checklists and automated CI verification using GitHub Actions were performed.

4. Requirement Engineering

Functional Requirements

The system shall:

- Allow registered students to log in.
- Allow logged-in students to start exams.
- Allow students to submit exams.
- Prevent multiple submissions of the same exam.
- Maintain active and submitted exam states.

Non-Functional Requirements

- The system shall maintain consistency.
- The system shall prevent invalid submissions.
- The system shall support automated validation.

Requirement Defect Taxonomy

Requirement	Defect Type	Description
Student login	Ambiguity	Login conditions were not clearly defined initially
Exam submission	Inconsistency	Submission and active exam states conflicted
Validation	Non-verifiable	Validation requirements lacked measurable criteria

GitHub Issue Tracking

GitHub Issues were used to track:

- Requirement corrections
 - Alloy verification errors
 - CI pipeline configuration issues
-

5. Z Modeling

Z Notation was used to formally model the system state and operations.

State Schema

The system state includes:

- Registered students
- Logged-in students
- Active exams
- Submitted exams

Invariants

Invariant 1

$\text{loggedIn} \subseteq \text{students}$

Only registered students are allowed to log in.

Invariant 2

$(s,e) \in \text{submitted} \Rightarrow (s,e) \notin \text{activeExam}$

A submitted exam cannot remain active.

Operations

The following operations were modeled:

- Login
- StartExam

- SubmitExam

These operations preserve system invariants and maintain state consistency.

The screenshot shows the Z-Editor interface with a blue header bar containing 'Z-Editor' and 'View on Github'. Below the header is a toolbar with icons for bold, italic, underline, and other text formatting, along with a font size dropdown set to 16 and a font family dropdown set to Arial. The main editing area contains the following VDM-SL code:

```
[STUDENT, EXAM]

OnlineExamSystem
==
students : P STUDENT
loggedIn : P STUDENT
activeExam : STUDENT * EXAM
submitted : STUDENT * EXAM

where
  loggedIn ⊆ students
  ∀ s: STUDENT; e: EXAM •
    (s,e) ∈ submitted ⇒ (s,e) ∉ activeExam

InitOnlineExamSystem
```

6. VDM-SL Specification

VDM-SL was used to define functional specifications.

System State

The system state contains:

- students
- loggedIn
- submitted

Preconditions

Examples:

- A student must exist before login.
- A student must be logged in before starting or submitting an exam.

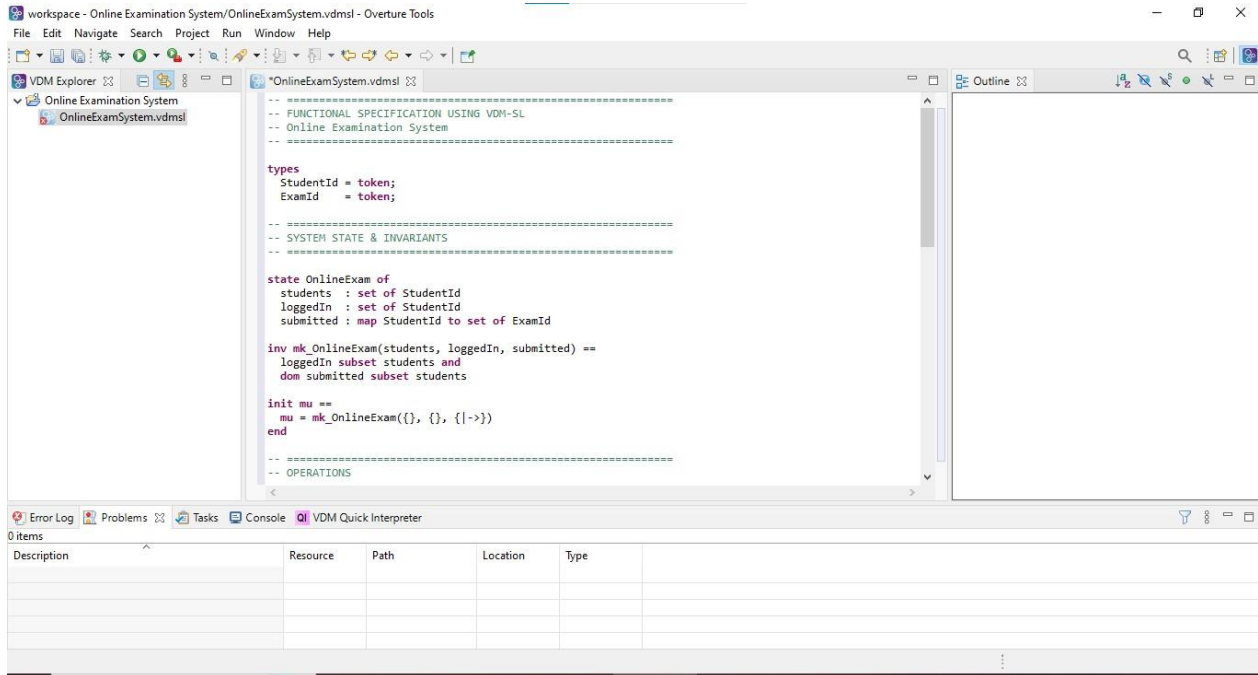
Postconditions

Postconditions ensure:

- Correct update of logged-in users.
- Consistent submission states.

Contract-Based Verification

The VDM specification verified that all operations satisfy defined contracts and preserve correctness.



7. Alloy Verification

Alloy Analyzer was used for structural verification.

Relational Model

The model contains:

- Student signatures
- Exam signatures
- Relations for active exams
- Relations for submitted exams

Constraints

The following constraint was enforced:

- A student cannot have the same exam in both active and submitted states.

Assertion Verification

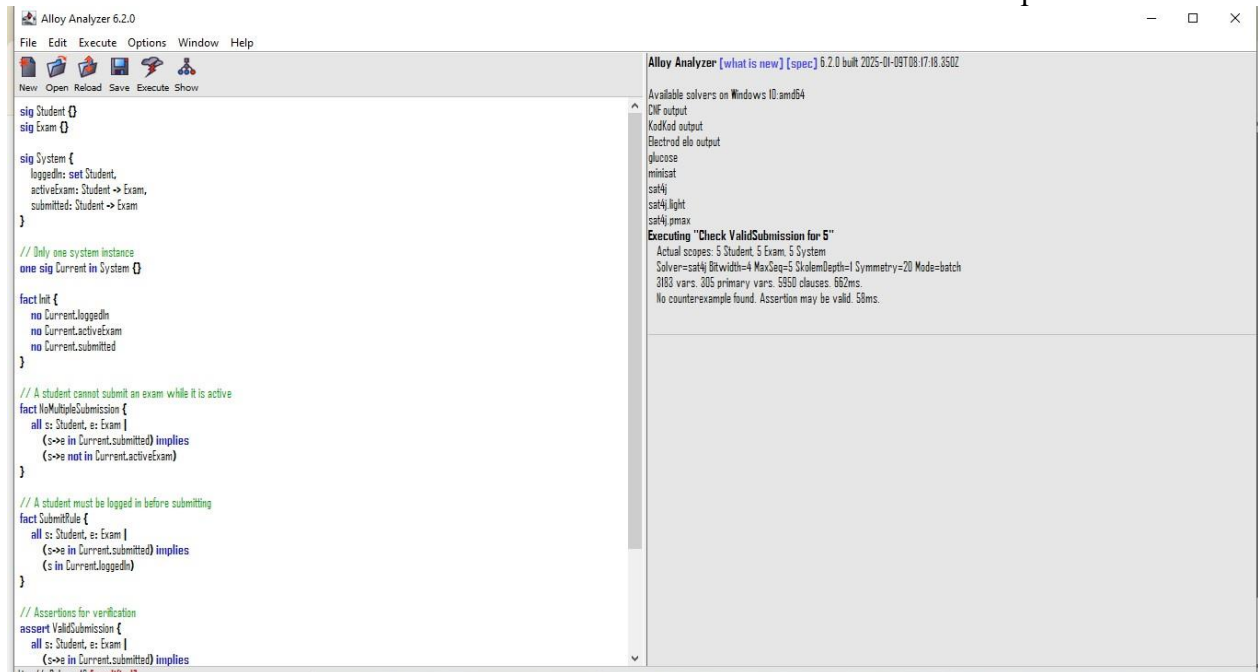
The assertion `ValidSubmission` was checked using Alloy Analyzer.

Verification Result

The analyzer produced the following result:

“No counterexample found. Assertion may be valid.”

This indicates that the defined constraint remained valid within the selected scope.



Counterexample Analysis

An intentionally incorrect constraint was added to generate counterexamples and demonstrate inconsistency detection in Alloy.

8. Validation

Validation Checklist

Validation Item	Status
Login verification	Passed
Exam submission verification	Passed
Invariant preservation	Passed
Alloy assertion validation	Passed
CI pipeline execution	Passed

github.com/Monam1083/Online-Examination-System-

<> Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

Online-Examination-System- Public

main 1 Branch 0 Tags

Go to file Add file <> Code

Monam1083 Update README.md ✓	a608922 · 9 minutes ago	2 Commits
☐ .github/workflows	first commit	10 minutes ago
☐ alloy-model	first commit	10 minutes ago
☐ ci-pipeline	first commit	10 minutes ago
☐ requirements	first commit	10 minutes ago
☐ validation	first commit	10 minutes ago
☐ vdm-spec	first commit	10 minutes ago
☐ z-model	first commit	10 minutes ago
☐ README.md	Update README.md	9 minutes ago

About

No description, website, or topics provided.

Readme Activity 1 star 0 watching 0 forks

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

GitHub Actions CI Pipeline

A GitHub Actions CI pipeline was configured to automatically perform validation whenever changes were pushed to the repository.

Pipeline Result

Validation Successful

On April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. [Review this update](#) and manage your preferences in your [GitHub account settings](#).

SVV CI Pipeline

✓ Add CI pipeline for validation on main branch #1 Re-run all jobs ...

Summary

All jobs

validation

Run details

Usage

Workflow file

Triggered via push now

chfurqan510r pushed · 7d0d6fb · main

Status: Success

Total duration: 10s

Artifacts: —

svv-ci.yml

on: push

validation 6s

9. Results

The project successfully demonstrated formal verification of an Online Examination System using multiple verification techniques and validation tools.

Key Achievements

- Formal modeling using Z Notation
- Functional specification using VDM-SL
- Structural verification using Alloy Analyzer
- Automated validation using GitHub Actions

All system invariants remained preserved throughout the modeled operations.

10. Conclusion

This project demonstrated the practical implementation of Software Verification and Validation techniques using formal methods. The Online Examination System was successfully modeled, verified, and validated using Z Notation, VDM-SL, Alloy Analyzer, and GitHub Actions.

The project proved that formal methods help identify inconsistencies and improve system correctness before implementation. Alloy verification confirmed structural consistency, while Z and VDM specifications ensured functional correctness and invariant preservation.

The project also highlighted the importance of automated validation in modern software engineering practices.